

DAY 21 — Production Database Project + Final SQL/PostgreSQL Assessment

Today is the final day of Phase 2.

No major new theory today. You will combine everything you've learned into one realistic production-style RAG database system, then complete an interview assessment.

Important: I will not give solutions before you attempt them. I'll review your SQL like a technical interviewer.

Day 21 Objectives

By the end of today, you should be able to:

- Design a normalized PostgreSQL schema
- Use PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT
- Write complex JOINS
- Correctly handle LEFT JOIN + NULL
- Use CTEs
- Use window functions
- Use aggregation and conditional aggregation
- Design useful indexes
- Reason about transactions and ACID
- Handle concurrent workers
- Read EXPLAIN
- Diagnose slow queries
- Identify row multiplication
- Optimize queries based on evidence
- Explain database design decisions in an interview

PART 1 — Production RAG Database Project

Imagine you're building the backend database for a multi-user RAG platform.

Users can:

1. Upload documents
2. Documents are processed
3. Documents are split into chunks
4. Chunks receive embeddings

5. Users ask questions
6. Conversations and messages are stored
7. Background workers process ingestion jobs

Your database must support both:

Application APIs

↓

PostgreSQL

↓

RAG ingestion

↓

Chunks

↓

Embeddings

↓

Retrieval

PART 2 — Database Schema

You will work with these tables.

1. users

users

id

name

email

country

account_status

created_at

Rules:

- id → primary key
- email → unique
- account_status → active / suspended / inactive
- created_at → default current timestamp

2. documents

documents

id

user_id

title

file_type

page_count

status

created_at

updated_at

Rules:

- id → primary key
- user_id → FK → users
- page_count > 0
- status:

uploaded

processing

processed

failed

3. document_chunks

document_chunks

id

document_id

chunk_number

content

token_count

created_at

Rules:

- id → primary key
- document_id → FK

- (document_id, chunk_number) should be unique
- token_count > 0

4. embeddings

embeddings

id

chunk_id

model_name

embedding

created_at

For this project, treat embedding simply as a stored value such as vector.

Rules:

- id → primary key
- chunk_id → FK
- one embedding per chunk per model

Therefore:

(chunk_id, model_name)

should be unique.

5. conversations

conversations

id

user_id

title

created_at

updated_at

6. messages

messages

id

conversation_id

role

content

created_at

role:

user

assistant

system

7. ingestion_jobs

ingestion_jobs

id

document_id

status

attempt_count

started_at

completed_at

created_at

Status:

queued

running

completed

failed

8. tags

tags

id

name

name should be unique.

9. document_tags

document_tags

document_id

tag_id

created_at

This represents:

documents N ↔ N tags

Primary key:

(document_id, tag_id)

PART 3 — Project Task A: Schema Design

You need to write the PostgreSQL DDL.

Create:

users

documents

document_chunks

embeddings

conversations

messages

ingestion_jobs

tags

document_tags

Your DDL must demonstrate:

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE
- CHECK
- DEFAULT
- composite primary key
- composite unique constraint
- sensible timestamp columns

Challenge

Decide appropriate ON DELETE behavior for the relationships.

For example:

FOREIGN KEY (...)

REFERENCES ...

ON DELETE ...

Don't blindly use CASCADE.

Explain your reasoning.

PART 4 — Project Task B: User Document Dashboard

Write a query returning one row per user:

user_id

name

document_count

total_pages

average_pages

processed_documents

failed_documents

documents_without_chunks

Requirements:

- Include users with zero documents
- document_count should be correct
- Don't accidentally turn the LEFT JOIN into an INNER JOIN
- Don't count NULL-generated rows
- Use conditional aggregation
- Handle users with no documents

Important

This is deliberately testing your Day 14 weakness:

LEFT JOIN + NULL semantics

PART 5 — Project Task C: Document Pipeline Health

Return:

document_id

title

chunk_count

embedding_count

latest_job_status

Include documents even when:

0 chunks

or

0 embeddings

You should think carefully about row multiplication.

For example:

1 document

100 chunks

3 embeddings per chunk

A naive:

documents

JOIN chunks

JOIN embeddings

can produce:

300 rows

for one document.

Your counts must not become:

300 chunks

300 embeddings

when the actual values are:

100 chunks

300 embeddings

PART 6 — Project Task D: Latest Ingestion Job

For every document, return its latest ingestion job.

Output:

document_id

title

job_id

status

created_at

Requirements:

- Use a window function

- Use ROW_NUMBER()
- One latest job per document
- If a document has no job, decide whether it should remain in the result

Think carefully about:

PARTITION BY
ORDER BY

PART 7 — Project Task E: Top Users

Find the top 5 users by total document pages.

Return:

user_id

name

document_count

total_pages

rank

Requirements:

- Aggregate first
- Then rank users
- Use a window function
- Handle ties correctly

Question:

Would you use:

ROW_NUMBER()

or

RANK()

or

DENSE_RANK()

?

Explain why.

PART 8 — Project Task F: Failed Documents

Find users who have:

at least 3 failed documents

Return:

user_id

name

failed_document_count

Use:

GROUP BY

HAVING

Do not solve this using a window function unless you have a specific reason.

PART 9 — Project Task G: Documents Above User Average

For every user, find documents whose page count is greater than that user's average page count.

Output:

user_id

document_id

title

page_count

user_average_pages

You should use a window function such as:

AVG(...) OVER (

PARTITION BY ...

)

You will probably need a CTE because of SQL's logical processing order.

PART 10 — Project Task H: Index Design

Assume:

users = 10 million rows

documents = 500 million rows

document_chunks = 5 billion rows

embeddings = 15 billion rows

The following API endpoint is slow:

SELECT

d.id,

d.title,

d.created_at

FROM documents d

WHERE d.user_id = \$1

```
AND d.status = 'processed'  
ORDER BY d.created_at DESC  
LIMIT 20;
```

Existing indexes:

PRIMARY KEY (documents.id)

INDEX (documents.user_id)

INDEX (documents.status)

Your task

Propose an initial index hypothesis.

Consider:

(user_id, status, created_at)

versus:

(user_id, created_at)

versus a partial index.

Explain your choice based on:

- selectivity
- status distribution
- ordering
- LIMIT
- number of documents per user
- write overhead

Don't just say "create an index."

Explain why.

PART 11 — Project Task I: EXPLAIN

You receive:

Seq Scan on documents

(cost=0.00..12500000.00 rows=50)

(actual time=0.100..4200.000 rows=52)

Answer:

1.

What is suspicious?

2.

What does:

rows=50

mean?

3.

What does:

actual rows=52

mean?

4.

What could cause:

estimated rows $\approx$ 50

actual rows $\approx$ 5,000,000

?

5.

What would you investigate next?

PART 12 — Project Task J: Transaction

Implement the following logical operation:

1. Create ingestion job

2. Mark document as processing

3. Commit

All three operations must succeed together.

If step 2 fails:

ROLLBACK

The database should not contain an orphan ingestion job.

Write the transaction.

PART 13 — Project Task K: Concurrent Workers

Suppose you have:

100 queued ingestion jobs

and:

10 workers

Each worker needs to claim a different job.

Explain how you would use:

SELECT ...

FOR UPDATE SKIP LOCKED

to prevent workers from claiming the same job.

Conceptually:

Worker 1 → Job 101

Worker 2 → Job 102

Worker 3 → Job 103

...

No duplicate claiming.

Interview follow-up

Why might:

SELECT ...

FOR UPDATE

without:

SKIP LOCKED

be less suitable for a high-throughput worker queue?

PART 14 — Production Debugging

Find the problems in this query:

```
SELECT
u.id,
u.name,
COUNT(*) AS failed_documents
FROM users u
LEFT JOIN documents d
ON d.user_id = u.id
WHERE d.status = 'failed'
GROUP BY u.id, u.name;
```

There are at least two important semantic problems.

Explain them.

Then rewrite the query so that:

- every user appears
- users with zero failed documents appear
- failed documents are counted correctly

PART 15 — Row Multiplication Debugging

A developer writes:

```
SELECT
```

```
d.id,  
COUNT(c.id) AS chunks,  
COUNT(e.id) AS embeddings  
FROM documents d  
LEFT JOIN document_chunks c  
ON c.document_id = d.id  
LEFT JOIN embeddings e  
ON e.chunk_id = c.id  
GROUP BY d.id;
```

For one document:

chunks = 100

embeddings = 300

But the query returns:

chunks = 300

embeddings = 300

Explain exactly why.

Then propose a correct query strategy.

Hint:

Think about pre-aggregating child tables before joining.

PART 16 — Database Design Interview

Answer these without searching.

Q1

Why is this schema problematic?

documents

id

user_name

user_email

title

...

if the same user owns 10,000 documents?

Q2

Explain:

1NF

2NF

3NF

using a practical example.

Q3

When would denormalization be reasonable?

Q4

Should every foreign key automatically receive an index?

Explain.

Q5

Why can adding too many indexes hurt a production system?

PART 17 — PostgreSQL Interview

Q6

What happens conceptually when PostgreSQL receives:

SELECT ...

Describe:

Parser

↓

Analyzer

↓

Planner

↓

Executor

Q7

Why can PostgreSQL choose a Sequential Scan even when an index exists?

Give at least 3 reasons.

Q8

What's the difference between:

EXPLAIN

and:

EXPLAIN ANALYZE

?

Q9

Why is EXPLAIN ANALYZE potentially dangerous on:

UPDATE

DELETE

INSERT

?

Q10

Explain:

Nested Loop

Hash Join

Merge Join

at a high level.

PART 18 — SQL Interview

Q11

Difference between:

WHERE

HAVING

?

Q12

Difference between:

COUNT(*)

COUNT(column)

especially after a LEFT JOIN.

Q13

Why does this change the semantics of a LEFT JOIN?

```
LEFT JOIN documents d
ON d.user_id = u.id
WHERE d.status = 'failed'
```

Q14

What is the difference between:

ROW_NUMBER

RANK

DENSE_RANK

?

Q15

Why can't you normally do:

```
SELECT
*,
ROW_NUMBER() OVER (...) AS rn
FROM documents
WHERE rn <= 3;
```

?

Q16

What is the NOT IN + NULL trap?

Q17

When would you choose:

CTE

subquery

JOIN

?

Q18

Why can this be dangerous?

JOIN documents d
JOIN document_chunks c
JOIN embeddings e
when calculating counts?

PART 19 — Final Explanation Challenge

Imagine you're interviewing for a Junior/Mid-level Data Engineer / GenAI Engineer role.

The interviewer asks:

"A RAG API endpoint suddenly became slow after the database grew from 50 million to 500 million documents. What would you do?"

Give me your answer as if you're in the interview.

A strong answer should discuss something like:

- 1. Confirm correctness**
- 2. Reproduce the slow query**
- 3. EXPLAIN**
- 4. EXPLAIN ANALYZE**
- 5. Estimated vs actual rows**
- 6. Existing indexes**
- 7. Data distribution**
- 8. Selectivity**
- 9. Join strategy**
- 10. Sort**
- 11. Composite/partial index hypothesis**
- 12. Statistics**
- 13. Table growth**
- 14. Write overhead**
- 15. Before/after measurement**

Don't simply say:

"I would add an index."

DAY 21 FINAL ASSESSMENT

Now your final assessment.

Do not answer everything at once.

We'll do this like a real interview.

Round 1 — SQL

Question 1 / 15

You have:

users

id

name

documents

id

user_id

status

Data:

users

1 Aarav

2 Emma

3 Liam

4 Sofia

documents

101 1 processed

102 1 failed

103 2 processed

104 2 processed

Write a query that returns:

user_id

name

total_documents

failed_documents

Expected result:

1 Aarav 2 1

2 Emma 2 0

3 Liam 0 0

4 Sofia 0 0

Constraints

Your query must:

- include users with zero documents
- use LEFT JOIN
- correctly handle NULLs
- avoid turning the LEFT JOIN into an INNER JOIN
- correctly count failed documents

Your turn.

Send only your SQL for Question 1 first. I'll review it, score it, and then give you Question 2.